CS 307 Report for Assignment 2

İde Melis Yılmaz 32400

November 2025

1 Overview

I implemented the simulator exactly as the project asks: each core runs in its own thread, keeps a Sorted Dispatcher Database (SDD) of tasks, schedules them with STCF, and occasionally steals or donates jobs to balance the load. Every task stores its remaining duration, cache heat, and original owner so `executeJob` can reset cache affinity whenever a migration happens. The whole thing compiles with the provided Makefile (`make sim`) without touching the given driver files.

2 SDD Data Structure

Each queue is a doubly linked list that I keep in ascending order of remaining duration. In `submitTask` (sdd.c) I allocate a node for the new task, lock the queue, walk forward while the current node finishes earlier than the newcomer, and insert the new node right before the first longer task. If I fall off the end I just append and update `tail`. Because the list is sorted, fetching the STCF job is trivial: `fetchTask` detaches `head` under the same mutex, so the owner thread always gets the shortest job in O(1). When thieves need work they call `fetchTaskFromOthers`, which removes `tail`, so they grab the longest job and minimize cache hit disruption for the victim core. This linked-list layout also makes it easy to print the queue in order and to reinsert unfinished jobs without copying arrays or doing heapify operations.

3 Concurrency Design

Every SDD owns a `pthread_mutex_t` and never try to hold more than one lock at a time. Because there is no nested locking, there is no way to form a cycle of waiting threads, so deadlock cannot happen. A thread grabs the queue's mutex whenever it touches list pointers or the size counter (submit, fetch, fetch-from-others, print) and release it immediately after the critical section.

The owner can only call `fetchTask` (head) while stealers can only call `fetchTaskFromOthers` (tail). Both operations take the same lock, so a task

can't be removed twice or reinserted while someone else is halfway through a removal. Likewise, `submitTask` updates both neighbors and the `size` field inside the lock, so no insertion gets lost, and `print_queue` holds the lock for the duration of its traversal, which keeps the snapshot consistent even if another thread is queued up to donate or steal.

I deliberately chose one mutex per queue. A single global lock would serialize every core and kill parallelism; on the other hand, per-node locks or lock-free algorithms would complicate the project and probably cost more than they save given the number of threads we run. The current granularity keeps work local to each SDD, so contention mostly boils down to the brief head and tail operations.

4 Load-Balancing Algorithm

Inside `processJobs` I monitor my local queue length. If it drops below `LOW_WATERMARK = 3`, I scan the global `processor_queues` array to find the heaviest queue whose size is above `HIGH_WATERMARK = 8`. If I find one I steal its tail via `fetchTaskFromOthers`. After every time slice I reinsert a partially executed task into my own queue; if that makes my queue exceed the high watermark I immediately donate one tail task to the lightest queue I can find. Shipping the longest job keeps the shorter tasks on that core warm in the cache while still transferring a meaningful chunk of work. Because tasks retain their `owner` pointer, the next `executeJob` call will notice a migration and reset `cache_warmed_up` to model a cold start on the new core.

5 Watermark Sensitivity

I experimented with a few values and settled on `LOW = 3`, `HIGH = 8`. If I lower HIGH too much the donators start shipping tasks constantly and everybody suffers from cold caches. If I put it too high, busy cores end up stockpiling work and idle cores spend cycles polling for something to do. Similarly, a very high LOW causes idle cores to steal even when they still have a couple of tasks queued, which just increases locking traffic, whereas a LOW of 0 lets them wait until the queue is literally empty. This combination leaves enough room for the STCF .

6 STCF Policy Discussion

STCF is nice here because it minimizes average turnaround time on each core and plays well with the task generator (lots of short jobs). Once a job gets the head, I know it is the shortest remaining one so giving it the next slice improves fairness between short and long tasks. The downside is that STCF keeps preempting long jobs, so in a real kernel we would pay context-switch costs and interactive bursts might still get delayed. CFS would be better in

terms of less context switch costs probably and that is why it is used in real OS currently.

7 Testing Summary

I rebuilt with `make sim` and ran the three provided sample files. After that I generated six additional inputs (8 cores, up to `~35` tasks per core) and ran them as well. Every run ended with "All tasks finished, joining threads", so there were no deadlocks, repeats, or lost work. The stress runs took between 22 and 46 seconds of wall time because of the simulated CPU sleeps, but the actual CPU time stayed under `0.11 s` and the process never went above `2 MB` . In these results queues stay sorted, locks protect the data structure, and the watermarks keep the cores busy without cpu waste in scheduling.